

Rust 异步运行时设计

杨子凡

Oct 02, 2026

Rust 异步编程的流行并非偶然。在高并发场景下，线程模型的上下文切换开销与 C10k 瓶颈使开发者不得不转向事件驱动架构。本文将带你手写一个极简的异步运行时，理解 Executor、Reactor 与调度策略的核心思想，并对比 tokio、async-std 与 smol 三种成熟方案。

1 背景：为什么需要异步运行时

当我们使用传统线程来处理并发请求时，每个连接往往需要一个线程来维持。线程的创建、调度与销毁都涉及昂贵的内核态上下文切换。面对数万并发连接时，系统可能耗尽内存或因频繁切换而导致吞吐量下降。Rust 虽然提供了 `std::thread`、`std::sync::mpsc` 与 `Mutex` 等原语，但在高并发下这些同步工具容易成为瓶颈。异步的本质是用「状态机 + 事件循环」来替代阻塞等待，让单线程或少量线程就能驱动大量并发任务。

2 异步运行时核心抽象

2.1 Future 与 Poll

在 Rust 中，所有异步操作都表现为实现了 `Future trait` 的类型。`Future` 的核心方法是 `poll`，它接收一个 `Context` 并返回 `Poll<T>`。当 `poll` 返回 `Poll::Ready(val)` 时，表明异步操作已完成；返回 `Poll::Pending` 则表示尚未就绪，需要等待外部事件唤醒。理解这一点对后续实现 `Executor` 至关重要。

```
1 use std::future::Future;
  use std::pin::Pin;
3 use std::task::{Context, Poll};

5 pub struct MyFuture { /* 内部状态 */ }

7 impl Future for MyFuture {
  type Output = i32;

9
  fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
11     // 1. 检查内部状态是否就绪
    if self.ready {
13         // 2. 返回就绪值
```

```
15         Poll::Ready(42)
16     } else {
17         // 3. 注册唤醒器，等待外部事件
18         cx.waker().wake_by_ref();
19         Poll::Pending
20     }
21 }
```

这段代码演示了 `poll` 的基本流程：检查状态、返回结果或挂起。注意，`wake_by_ref` 会通知运行时该任务可再次被调度。

2.2 Waker 与任务唤醒

Waker 是连接 Future 与 Executor 的桥梁。当 Reactor 检测到 I/O 事件时，它会调用与该事件关联的 Waker 来唤醒对应任务。Waker 的实现通常包含一个指向任务队列的引用或索引，以便 Executor 能迅速找到待执行任务。

2.3 Executor 与 Reactor 的职责划分

Executor 负责管理任务队列与调度策略，而 Reactor 则专注于监听 I/O 事件并通过多路复用机制（如 `epoll`、`kqueue` 或 `IOCP`）高效分发。两者通过 Waker 进行协作：Reactor 产生事件，Executor 消费事件并驱动 Future 状态机。

3 手写一个最小运行时

3.1 项目结构与依赖

我们先创建一个仅依赖 `std` 与 `libc` 的项目，命名为 `minirt`。目录结构如下：

```
1 minirt/
  |—— Cargo.toml
3  |—— src/
  |   |—— main.rs
  |   |—— executor.rs
  |   |—— reactor.rs
```

`Cargo.toml` 中仅声明 `libc` 以便调用系统调用。

3.2 实现极简的 `block_on`

`block_on` 是运行时的入口，它会驱动一个 Future 直到完成。

```
use std::future::Future;
```

```

2 use std::pin::Pin;
  use std::task::{Context, Poll, RawWaker, RawWakerVTable, Waker};
4 use std::sync::Arc;

6 pub fn block_on<F: Future>(mut fut: F) -> F::Output {
    // 1. 创建一个简单的 Waker，它什么也不做
8    let waker = unsafe { Waker::from_raw(dummy_raw_waker()) };
    let mut cx = Context::from_waker(&waker);
10   // 2. 将 Future 固定在栈上
    let mut fut = unsafe { Pin::new_unchecked(&mut fut) };
12   // 3. 循环调用 poll，直到 Ready
    loop {
14       match fut.as_mut().poll(&mut cx) {
           Poll::Ready(val) => return val,
16       Poll::Pending => {
           // 4. 在单线程示例中，直接让出 CPU
18           std::thread::yield_now();
           }
20     }
22 }

24 // 省略 dummy_raw_waker 的实现

```

这段代码展示了 `block_on` 的核心逻辑：不断调用 `poll`，直到 `Future` 返回就绪值。在真实场景中，`yield_now` 会被更高效的事件等待替代。

3.3 任务队列

`Executor` 需要维护一个任务队列。单线程版本可使用 `VecDeque`，跨线程版本则需使用 `crossbeam::deque` 或 `std::sync::mpsc`。下面是一个基于 `VecDeque` 的简单实现：

```

use std::collections::VecDeque;
2 use std::future::Future;
  use std::pin::Pin;
4 use std::task::{Context, Poll};

6 pub struct Executor {
    tasks: VecDeque<Pin<Box<dyn Future<Output = ()>>>>,
8 }

```

```

10 impl Executor {
    pub fn new() -> Self {
12         Executor { tasks: VecDeque::new() }
    }

14
    pub fn spawn<F: Future<Output = ()> + 'static>(&mut self, fut: F) {
16         self.tasks.push_back(Box::pin(fut));
    }

18
    pub fn run(&mut self) {
20         while let Some(mut task) = self.tasks.pop_front() {
            // 创建一个 dummy waker, 实际使用时需替换
22             let waker = unsafe { Waker::from_raw(dummy_raw_waker()) };
            let mut cx = Context::from_waker(&waker);
24             if task.as_mut().poll(&mut cx) == Poll::Pending {
                self.tasks.push_back(task);
26             }
        }
28     }
}

```

spawn 将 Future 装箱后推入队列，run 则依次取出并驱动，直到所有任务完成或再次挂起。

3.4 Reactor: 用 epoll 监听事件

Reactor 的职责是监听文件描述符上的 I/O 事件。以 Linux 为例，我们使用 epoll。

```

1 use libc::{self, epoll_event, EPOLLIN};
use std::os::unix::io::RawFd;

3
pub struct Reactor {
5     epoll_fd: RawFd,
}

7
impl Reactor {
9     pub fn new() -> std::io::Result<Self> {
        // 1. 创建 epoll 实例
11         let epoll_fd = unsafe { libc::epoll_create1(0) };
        if epoll_fd < 0 {
13             return Err(std::io::Error::last_os_error());
        }
    }
}

```

```

15     Ok(Reactor { epoll_fd })
16 }
17
18 pub fn register(&self, fd: RawFd, waker: Waker) -> std::io::Result<()> {
19     // 2. 将 fd 及其关联的 waker 注册到 epoll
20     let mut ev = epoll_event {
21         events: EPOLLIN as u32,
22         u64: Box::into_raw(Box::new(waker)) as u64,
23     };
24     let ret = unsafe {
25         libc::epoll_ctl(self.epoll_fd, libc::EPOLL_CTL_ADD, fd, &mut ev)
26     };
27     if ret < 0 {
28         Err(std::io::Error::last_os_error())
29     } else {
30         Ok(())
31     }
32 }
33
34 pub fn poll(&self, timeout_ms: i32) -> std::io::Result<Vec<Waker>> {
35     // 3. 等待事件, 最多返回 128 个
36     let mut events = [epoll_event { events: 0, u64: 0 }; 128];
37     let n = unsafe {
38         libc::epoll_wait(self.epoll_fd, events.as_mut_ptr(), 128, timeout_ms)
39     };
40     if n < 0 {
41         return Err(std::io::Error::last_os_error());
42     }
43     // 4. 提取 waker 并唤醒
44     let mut wakers = Vec::new();
45     for ev in &events[..n as usize] {
46         let waker = unsafe { Box::from_raw(ev.u64 as *mut Waker) };
47         waker.wake();
48         wakers.push(*waker);
49     }
50     Ok(wakers)
51 }
52 }

```

register 将文件描述符与 Waker 绑定, poll 则阻塞等待事件并唤醒对应任务。

3.5 把 Future 状态机与 epoll 事件关联

当一个异步 TCP 连接需要等待数据时，Future 会把自身的 Waker 注册到 Reactor。数据到达后，Reactor 通过 epoll 事件唤醒 Future，Future 再次被 Executor 调度。

3.6 一次完整的 echo server 请求生命周期

- 服务器启动时创建监听 socket 并注册到 Reactor。
- 客户端连接到达，Reactor 产生 EPOLLIN 事件，唤醒 accept Future。
- accept Future 返回新连接，Executor 将 echo Future 加入队列。
- echo Future 尝试 read，若无数据则把 Waker 注册到 Reactor 并返回 Pending。
- 数据到达，Reactor 唤醒 echo Future，Future 继续执行并写回响应。

4 调度策略与性能权衡

4.1 Work-stealing 与 Global queue

Work-stealing 调度器为每个线程维护一个本地队列，空闲线程可从其他线程「偷取」任务，降低全局锁竞争。Global queue 则所有线程共享一个队列，简单但易成为瓶颈。

4.2 优先级与饥饿

为避免低优先级任务长期得不到执行，调度器通常引入优先级或 aging 机制。tokio 通过注入「LIFO 插队」与「公平轮转」来平衡响应延迟与吞吐量。

4.3 批量 poll 与 spin-loop 阈值

连续多次 poll 可能导致 CPU 空转。运行时会设置一个「spin-loop」阈值，超过后让出 CPU 或进入休眠，等待 Reactor 事件。

4.4 NUMA 与 CPU 亲和性

在 NUMA 架构下，跨节点访问内存代价高昂。tokio 提供了 `tokio::runtime::Builder::on_thread_park` 等钩子，允许用户设置 CPU 亲和性，减少跨节点通信。

5 成熟运行时对比

tokio 采用 work-stealing 调度与 mio 后端，拥有最成熟的生态与零拷贝支持。async-std 同样使用 work-stealing，但更强调与标准库一致的 API。smol 则定位为轻量级方案，默认单线程，适合嵌入式或教学场景。我们手写的 minirt 仅实现 FIFO 与 epoll，缺乏定时器与零拷贝，生态成熟度最低，但足以展示核心原理。

6 工程化考量

6.1 取消与超时

`select!` 宏可同时驱动多个 `Future`，任意一个完成即取消其余。`timeout` 则在指定时间内未完成即返回错误，二者都依赖 `Waker` 的正确注册与注销。

6.2 背压与流量控制

异步 `channel` 的容量直接影响背压。若发送方速度远快于接收方，队列可能耗尽内存。合理设置容量并在满时阻塞发送方，可避免系统雪崩。

6.3 内存模型与 `Pin`

`Pin` 保证 `Future` 在 `poll` 期间不会被移动，从而安全使用自引用结构。开发者需谨慎使用 `Box::pin` 与 `Pin::new_unchecked`，避免悬垂指针。

6.4 调试

`tracing crate` 可记录任务生命周期，`tokio-console` 提供实时任务视图，火焰图则帮助定位 CPU 热点。

7 案例研究：把最小运行时扩展为支持多线程

7.1 引入全局/局部队列

多线程版本为每个线程分配一个本地 `Worker`，并共享一个全局 `Injector`。当本地队列为空时，线程先尝试从全局队列窃取任务，再尝试从其他线程窃取。

7.2 无锁并发数据结构

`crossbeam::deque` 提供无锁双端队列，支持 `steal` 与 `push` 的并发操作，避免了全局锁。

7.3 线程池初始化与优雅退出

在 `main` 中创建固定数量的线程，每个线程运行 `Worker::run`。当收到关闭信号时，线程会完成当前任务后退出，确保资源正确释放。

8 未来与生态

8.1 `io_uring` 与高性能内核旁路

`io_uring` 允许用户态与内核通过共享内存提交与完成 I/O 请求，减少系统调用开销。Rust 生态已有 `io_uring crate`，可作为 `Reactor` 的高性能后端。

8.2 异步与 `async fn` 语法稳定化

`async fn` 与 `await` 已成为稳定语法，Future 状态机由编译器自动生成，极大降低了异步编程的心智负担。

8.3 跨语言运行时互操作

C# 的 `Task`、Python 的 `asyncio` 与 Rust 的 `Future` 可通过 FFI 或消息队列实现互操作，构建混合语言的高并发服务。

通过手写 `minirt`，我们理解了 `Executor`、`Reactor` 与调度策略的协作方式。下一步可尝试实现带优先级的调度、支持超时的 `timeout` 以及接入 `io_uring`，以进一步提升性能。推荐阅读 *The tokio-rs book*、*Programming Rust* 第 2 版中的「Rust Concurrency」章节，以及 `epoll` 与 `io_uring` 的官方文档。